COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithm	**SECTION:**	Kenji Baltazar
DATE PERFORMED:	08/04/2026	**DATE SUBMITTED:**	08/011/2026
NAME:	Kenji Baltazar	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 5.1 **Seatwork 5.1: Progress Report Hands-on Activity 5.1 Queues**

1. PROCEDURE OUTPUT

ILO A: Create queue using C++ STL

QUEUE STL CODE

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> r);

int main(){
    std::queue<char> q;

    q.push('C');
    q.push('P');
    q.push('E');
    q.push('2');
    q.push('1');
    q.push('S');
    q.push('1');

    //display
    display(q);
    std::cout<<"\n";
    //empty
    if(q.empty()){
        std::cout<<"Queue is empty\n";
    } else{
        std::cout<<"Queue is not empty\n";
    }
```

```
26      //size
27      std::cout<<"Size of Queue is: "<<q.size()<<"\n";
28      //front
29      std::cout<<"The front of the queue is: "<<q.front()<<"\n";
30      //back
31      std::cout<<"The back of the queue is: "<<q.back()<<"\n";
32      //pop
33      std::cout<<"Popping out "<<q.front()<<"...\n";
34      q.pop();
35      //display
36      display(q);
37      std::cout<<"\n";
38      //push another element
39      q.push('0');
40      //display
41      display(q);
42      std::cout<<"\n";
43      //delete the queue
44          while(!q.empty()){
45          q.pop();
46      }
47      //check if empty
48      if(q.empty()){
49          std::cout<<"Queue is empty\n";
50      } else{
51          std::cout<<"Queue is not empty\n";
52      }
53
54      return 0;
55  }
56
57  void display(std::queue<char> r){
58      //create a copy of the queue
59      std::queue<char> c = r;
60
61      while(!c.empty()){
62          std::cout<<" "<<c.front();
63          c.pop();
64      }
65  }
66
```

OUTPUT

```
 C P E 2 1 S 1
Queue is not empty
Size of Queue is: 7
The front of the queue is: C
The back of the queue is: 1
Popping out C...
 P E 2 1 S 1
 P E 2 1 S 1 0
Queue is empty
```

Code Analysis: As I examined and tried each STL queue functions I observed that it is just like doing the stack implementation where:
-pushing adds an element of your choice to the end of the queue
-empty checks if your present queue is empty or not
-size checks all the element currently in your queue
-front is like the top in stack where it checks the first element on our queue
-back is to check what is the last element of our queue
In addition, I also implemented a function to display all the elements in our queue by using a while loop and also using empty function as a condition so that we could loop until all the elements in the loop empties out to end.

B.1. Linked List Implementation

QUEUE LINKED LIST CODE *(main.cpp)*

```
#include <iostream>
#include "queuell.h"

void display(qNode<char>** front, qNode<char> back);


void display();
int main(){
    qNode<char> *front = nullptr;
    qNode<char> *back = nullptr;

    enqueue(&front, &back, 'K');
    std::cout<<front->data<<" "<<back->data<<std::endl;
    enqueue(&front, &back, 'E');
    std::cout<<front->data<<" "<<back->data<<std::endl;
    enqueue(&front, &back, 'N');
    std::cout<<front->data<<" "<<back->data<<std::endl;
    dequeue(&front, &back);
    std::cout<<front->data<<" "<<back->data<<std::endl;

    //print all
    std::cout<<"Displaying all... ";
    display(front);
    //return the front variable
    std::cout <<"Front value: " << returnFront(front);

    return 0;
}

```

QUEUE LINKED LIST CODE *(queuell.h)*

```cpp
#ifndef QUEUELL_H
#define QUEUELL_H

#include <iostream>
template <typename T>
class qNode{
    public:
    T data;
    qNode *next;
};

//creating a new node
template <typename T>
qNode<T> *new_node(T newData){
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

template <typename T>
void enqueue(qNode<T>** frontPtr, qNode<T>** backPtr, T newData){
    //create a new node
    qNode<T>* newNode = new_node(newData);

```

```
26      //inserting to an empty queue
27      if((*frontPtr) == nullptr && (*backPtr) == nullptr){
28          (*frontPtr) = newNode;
29          (*backPtr) = newNode;
30      }
31
32      //inserting an item into a non empty queue
33      //point the backPtr next to the newNode
34      (*backPtr)->next = newNode;
35      (*backPtr) = newNode;
36  }
37
38  template <typename T>
39  void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
40      //create a temp node to store the node to be deleted
41      qNode<T>* deleteNode = nullptr;
42      deleteNode = (*frontPtr);
43
44      //check if the queue is only 1 node
45      if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
46          (*frontPtr) = nullptr;
47          (*backPtr) = nullptr;
48          delete deleteNode;
49          return;
50      }
```

```
51      //deleting of the node
52      (*frontPtr) = deleteNode->next;
53      delete deleteNode;
54  }
55
56  template <typename T>
57  void display(qNode<T>* front){
58      if(front == nullptr){
59          std::cout<<"Empty Queue\n";
60          return;
61      }
62      while(front != nullptr){
63          std::cout<<front->data<<" ";
64          front = front->next;
65      }
66      std::cout<<"\n";
67  }
68
69  template <typename T>
70  T returnFront(qNode<T>* frontPtr){
71      if(frontPtr != nullptr ){
72          return frontPtr->data;
73      }
74  }
75
76
77  #endif
```

B.2. Array-Based Implementation

QUEUE ARRAY CODE *(main.cpp)*

```cpp
#include <iostream>
#include "queuell.h"
#include "queueArr.h"

int main() {

    queueArr<int> q(6);
    q.Enqueue(1);
    std::cout<<"q.front: " << q.Front() << " q_back: " << q.Back() << std::endl;
    q.Enqueue(2);
    std::cout<<"q.front: " << q.Front() << " q_back: " << q.Back() << std::endl;
    q.Enqueue(3);
    std::cout<<"q.front: " << q.Front() << " q_back: " << q.Back() << std::endl;
    q.Enqueue(4);
    std::cout<<"q.front: " << q.Front() << " q_back: " << q.Back() << std::endl;
    q.Enqueue(5);
    std::cout<<"q.front: " << q.Front() << " q_back: " << q.Back() << std::endl;
    q.Enqueue(6);
    std::cout<<"q.front: " << q.Front() << " q_back: " << q.Back() << std::endl;
    q.Enqueue(6);
    q.Dequeue();
    std::cout<<"q.front: " << q.Front() << " q_back: " << q.Back() << std::endl;
    q.Dequeue();
    std::cout<<"q.front: " << q.Front() << " q_back: " << q.Back() << std::endl;
    q.Dequeue();
```

```
26      std::cout<<"q.front: " << q.Front() << " q_back: " << q.Back() << std::endl;
27      q.Dequeue();
28      std::cout<<"q.front: " << q.Front() << " q_back: " << q.Back() << std::endl;
29      q.Dequeue();
30      std::cout<<"q.front: " << q.Front() << " q_back: " << q.Back() << std::endl;
31      q.Enqueue(1);
32
33      q.Clear();
34      std::cout<< q.Front() << "\n" << q.Back();
35      std::cout<<"\n";
36
37      //for copy constructor
38      queueArr<int> q2=q;
39
40      //for assignment operator
41      queueArr<int> q3(5);
42      q3 = q;
43
44      return 0;
45  }
```

QUEUE ARRAY CODE *(queueArr.h)*

```
#ifndef QUEUEARR_H
#define QUEUEARR_H

#include <iostream>

//Array-Based Circular Queue
template <typename T>
class queueArr{
    private:
        //create a pointer array:
        T* q_array;
        size_t q_capacity;
        size_t q_size;
        int q_front;
        int q_back;
    public:
    //prototype
    queueArr(size_t capacity);
    bool Empty();
    bool Full();

    size_t Size();
    void Clear();

    T Front();
    T Back();

    void Enqueue(T newData);
    void Dequeue();

```

```
31      ~queueArr();
32
33      //taskss
34      queueArr(const queueArr& other);
35      queueArr& operator = (const queueArr& other);
36  };
37
38  //constructor
39  template <typename T>
40  queueArr<T>::queueArr(size_t capacity){
41      //capacity
42      q_capacity = capacity;
43      //current size
44      q_size = 0;
45      //initial front & back index
46      q_front = 0;
47      q_back = -1;
48      //newly allocated array
49      q_array = new T[q_capacity];
50      std::cout<<"constructor class was called\n";
51  }
52
53  //copy constructor
54  template <typename T>
55  queueArr<T>::queueArr(const queueArr& other){
56      //copy
57      q_capacity = other.q_capacity;
58      q_size = other.q_size;
59      q_front = other.q_front;
60      q_back = other.q_back;
```

```
61
62      //allocate a new memory for array
63      q_array = new T[q_capacity];
64
65      //copy the elements from the other array to this brand new array
66      for(size_t i = 0; i < q_size; ++i){
67          size_t index = (q_front + i) % q_capacity;
68          q_array[index] = other.q_array[index];
69      }
70      std::cout<<"copy constructor was called\n";
71  }
72
73  //copy assignment operator
74  template <typename T>
75  queueArr<T>& queueArr<T>::operator=(const queueArr& other){
76      //check if curr object is not the same the other one
77      if(this != &other){
78          delete [] q_array;
79          q_capacity = other.q_capacity;
80          q_size = other.q_size;
81          q_front = other.q_front;
82          q_back = other.q_back;
83
84          //allocate again
85          q_array = new T[q_capacity];
86
87          //copy again
88          for(size_t i = 0; i < q_size; ++i){
89              size_t index = (q_front + i) % q_capacity;
90              q_array[index] = other.q_array[index];
```

```
91              }
92              std::cout<<"copy assignment operator was called\n";
93          }
94          return *this;
95      }
96
97      //destructor()
98      template <typename T>
99      queueArr<T>::~queueArr(){
100         delete [] q_array;
101         std::cout<<"destructor was called\n";
102     }
103
104     //empty()
105     template <typename T>
106     bool queueArr<T>::Empty(){
107         return q_size == 0;
108     }
109
110     //full()
111     template <typename T>
112     bool queueArr<T>::Full(){
113         return q_size == q_capacity;
114     }
115
116     //clear()
117     template <typename T>
118     void queueArr<T>::Clear(){
119         q_front = 0;
120         q_back = -1;
```

```
121     q_size = 0;
122 }
123
124 //front()
125 template <typename T>
126 T queueArr<T>::Front(){
127     //check  & display if the q is empty
128     if(Empty()){
129         std::cout<<"The queue is empty\n";
130         return T{}; //std::optional
131     }
132     //return the front item
133     return q_array[q_front];
134 }
135
136 //back()
137 template <typename T>
138 T queueArr<T>::Back(){
139     //check  & display if the q is empty
140     if(Empty()){
141         std::cout<<"The queue is empty\n";
142         return T{}; //std::optional
143     }
144     //return the back item
145     return q_array[q_back];
146 }
147
148 //enqueue()
149 template <typename T>
150 void queueArr<T>::Enqueue(T newData){
```

```
151         //check if the queue is full
152         if(Full()){
153             std::cout << "Queue is full" << std::endl;
154             return;
155         }
156         //compute for the q_back to move it circularly
157         q_back = (q_back + 1) % q_capacity;
158 
159         //assign the new value to the back of the queue array
160         q_array[q_back] = newData;
161         //increment the q_size
162         q_size++;
163     }
164 
165     //dequeue()
166     template <typename T>
167     void queueArr<T>::Dequeue(){
168         //check if its empty
169         if(Empty()){
170             std::cout<<"The queue is empty\n";
171             return;
172         }
173         //compute for q_front to move it circulary
174         q_front = (q_front + 1) % q_capacity;
175 
176         //decrement the q_size
177         q_size--;
178     }
179 
180     template <typename T>
181     size_t queueArr<T>::Size(){
182         return q_size;
183     }
184 
185     #endif //QUEUEARR_H
```

OUTPUT

```
constructor class was called
q.front: 1 q_back: 1
q.front: 1 q_back: 2
q.front: 1 q_back: 3
q.front: 1 q_back: 4
q.front: 1 q_back: 5
q.front: 1 q_back: 6
Queue is full
q.front: 2 q_back: 6
q.front: 3 q_back: 6
q.front: 4 q_back: 6
q.front: 5 q_back: 6
q.front: 6 q_back: 6
The queue is empty
0
The queue is empty
0
copy constructor was called
constructor class was called
copy assignment operator was called
destructor was called
destructor was called
destructor was called
```

Code Analysis: As per usual, in our header file, we made our class and templated it, initializing variables we are gonna use in private whilst prototyping our function in public.

- Constructor, this is where we initialize our queue with a fixed size as what we are doing is an array. Now why are we using -1 instead of 0 for q_back? If we use 0 instead, our first enqueue would skip past index 0 and place the value on index 1. Why? Because enqueuing always adds 1 before we even place our value.
- Copy constructor, this is where we create a new object by initializing it from an existing object. By first copying all of the values of our existing object to our new object, then we now allocate a new memory for our array, then we need to copy the elements from the other array to a brand new array. How? By just making a loop that depends on q_size, which is to run how many times, then we need to compute using the modulo to copy only the right elements in proper order.
- Copy assignment operator, almost the same thing as the copy constructor, most of the lines were identical to our copy constructor, difference is we first check if our current object and the other object is not the same because we need to avoid deleting our own data, then we free the existing array before

allocating a new one because unlike copy constructor, the object has already a memory from before.
- Destructor is just for freeing these dynamic allocated arrays.
- Full and Empty functions are just to check if the q_size has reached our max capacity or if the q_size is 0, which returns either true or false.
- Clear, the main purpose of this function is to reset our queue back to when it was first constructed. We do this just by equating q_front, back, and size to its default value from our constructor function.
- Front and Back functions are just to return either front and back of our queue, but before that we need to check if our queue is empty or not before actually returning a value.
- Enqueue, this is just like the push function where we first need to check if our queue is full so that we could start pushing a value into the queue, then to make our array circular, we use modulo to advance our q_back circularly. The logic here is once (q_back + 1) and q_capacity are both the same number the last index will be calculated by modulo or the remainder, then when both are divided to one another, it will be 0, there is no remainder. Then now we can add a new value to the back of our array. Then increase our size.
- Dequeue, this is for removing the value from the front of our array where we need to check first if our queue is empty so that we could remove a value. Then similar to enqueue, we use the modulo to advance our q_front circularly, and we decrease our size.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

SUPPLEMENTARY CODE

```cpp
#include <iostream>
#include <string>
#include "queueArr.h"

class Job {
    public:
        int id;
        std::string userName;
        int pageNum;
        //prototypes
        Job();
        Job(int jobId, std::string name, int pages);
};

class Printer {
    private:
        queueArr<Job>* jobb;

    public:
        //constructor
        Printer(int capacity){
            jobb = new queueArr<Job>(capacity);
        }
        //destructor
        ~Printer(){
            delete jobb;
        }
        //prototypes
        void add(Job addJob);
        void process();
```

```
};

int main() {
    Printer p(5); //The limit of what our printer can print at a time
    std::cout << "Zoro was about to send his file but he was too late\n\n";
    //          ID      NAME   PAGE
    p.add(Job(2515531, "Kenji", 6));
    p.add(Job(2345678, "Bakugo", 1));
    p.add(Job(2000123, "Shoto", 7));
    p.add(Job(2582791, "Might", 3));
    p.add(Job(2397208, "Deku", 5));
    p.add(Job(9999999, "Zoro", 4)); //Zoro cannot print because it is full
    p.process(); //now processing the jobs

    std::cout << "\nNow Zoro was able to send his file this time\n\n";
    p.add(Job(9999999, "Zoro", 4)); //he can send his file again
    p.process();

    return 0;
}
//function to make a job but default
Job::Job(){
    id = 0;
    userName = "";
    pageNum = 0;
}
//function to create a job with parameters
Job::Job(int jobId, std::string name, int pages){
    id = jobId;
    userName = name;
```

```
61      pageNum = pages;
62  }
63  //function to add job to our queue
64  void Printer::add(Job addJob){
65      //checking if our queue is full or not
66      if(jobb->Full()){
67          std::cout << addJob.userName << " cannot print! Queue is full.\n";
68          return;
69      }
70      //now we can add job to our queue
71      jobb->Enqueue(addJob);
72      std::cout << addJob.userName << " sent " << addJob.pageNum << " paged file.\n";
73  }
74  //function to process jobs to queue
75  void Printer::process(){
76      //check if queue is empty or not
77      if(jobb->Empty()){
78          std::cout << "No jobs to process!\n";
79          return;
80      }
81      //now we process the queue
82      while(!jobb->Empty()){
83          Job current = jobb->Front(); //we first get the front most job
84          jobb->Dequeue(); //then we remove it from our queue to simulate processing
85          std::cout << "Printing " << current.userName << "'s file" << "...done!\n";
86      }
87  }
```

OUTPUT

```
constructor class was called
Zoro was about to send his file but he was too late

Kenji sent 6 paged file.
Bakugo sent 1 paged file.
Shoto sent 7 paged file.
Might sent 3 paged file.
Deku sent 5 paged file.
Zoro cannot print! Queue is full.
Printing Kenji's file...done!
Printing Bakugo's file...done!
Printing Shoto's file...done!
Printing Might's file...done!
Printing Deku's file...done!

Now Zoro was able to send his file this time

Zoro sent 4 paged file.
Printing Zoro's file...done!
destructor was called
```

Output Analysis: As required in the steps, I did not use STL application and used array implementation instead. Why array instead of linked list? Well it is so that I can know the limit of the maximum capacity of the printer, if I used linked list instead, I wouldn't really know the limits of it because it is unlimited until the memory persists.

Well onto the code, why does my output make sense? Because printers have limits and have an order first come first served. As seen on my output, we six people sent our file to the printer but it has a limit of 5, so the first five went in alright but the sixth one "Zoro" cannot print as it was already full. This is the full function in effect from the header file that I used which is from the procedure. Then next, I process the files by using a function, this was done by first getting the front most job and calling the dequeue function. And as seen on the output, the jobs were printed in order it was sent, First In-First Out. Then why can Zoro send his file again after the first five have been processed? It is because once the process is done, all will be cleared out because we have a circular array where we can reuse the same block of memory over and over again.

CONCLUSION:

In this activity, I learned about queue, that it is a FIFO, First-In First Out method, and that linked lists are better for no limit queues and that arrays can be better if limit is wanted. Making one function at a time really helped me understand the queue a lot more, as I delve in more and code more functions I became more aware of what code to do next or so. It also took me a lot of time to realize the patterns for the header files, but somehow I am starting to get it. Overall, I think I understood the main details or the main ideas of queue and also making a header file for

array and linked list, especially the circular index part by using the modulo. But I think I need to improve more as I am lacking in overall aspects, though I know the concepts, I keep blanking out of what code to do what, even though I know the methods already, I also need to face pointers more as this is the main priority in doing linked lists, coding more and also not just memorizing but familiarizing myself will all concepts like the BIG 3, the insertion up to now will also really help me generally, and also help me in the upcoming examination.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance in supplementary**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Claude was used only to assist me and verify my own queue implementation, the analysis, the original concept, and the full completion of my work was the work of my own.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.